

The vox Ratchet Tools: A Z Specification

Formal model of check-oo, check-coupling, check-suppressions

July 2026

Contents

1	Introduction	2
2	Basic Types	2
3	Free Types	2
4	Metric Ordering	2
5	Metric Maps	3
6	The Shared Fail-Closed Core	4
6.1	The git environment	4
6.2	Base unresolvable	4
6.3	Base resolves but carries no baseline blob	5
6.4	Base baseline blob corrupt	5
6.5	Reaching the comparison	5
6.6	The CI-write guard	5
6.7	Fail-closed properties	6
7	check-oo: The Must-Improve Ratchet	6
7.1	The empty-baseline intercept	6
7.2	The audited waiver	6
7.3	Inputs	7
8	check-coupling: The Regression-Only Ratchet	8
8.1	The empty-baseline intercept	8
8.2	The comparison	9
9	check-suppressions: The Scalar-Count Ratchet	9
10	The Never-Loosen Update	10
10.1	The update operation	10
10.2	The never-loosen property	11
10.3	The suppression update	11
11	Findings	12
12	System Invariants	12

1 Introduction

Vox enforces three code-quality ratchets, each a command-line gate run under `make check` and in CI:

- **check-oo** (`tools/oo_ratchet`): a *must-improve* ratchet on per-file object-orientation metrics.
- **check-coupling** (`tools/coupling`): a *regression-only* ratchet on per-file coupling/cohesion metrics.
- **check-suppressions** (`tools/suppression`): a *regression-only* ratchet on a single scalar — the total count of lint/type suppression comments plus **per-file-ignores**.

PR #312 unified all three onto one *fail-closed*, *base-commit-authoritative* core. They differ only in what they compare. This document therefore models the shared core once — base resolution, the fail-closed transition set, the CI-write guard, and the never-loosen update contract — and then adds one operation schema per tool for the part that genuinely differs.

The specification type-checks under `fuzz -t`. Property schemas restate the load-bearing guarantees — fail-closed, never-loosen, base-commit-authoritative — as type-checked predicates over the transition schemas. All three hold uniformly across the three tools; section 11 records how the last three per-tool exceptions were closed.

2 Basic Types

The three carriers the model needs no internal structure for: repository relative Python file paths, metric names, and git commit hashes.

$$[FILE, METRIC, COMMIT]$$

3 Free Types

The Boolean type: `fuzz` has none built in, and `ZBOOL` avoids the `BOOL/true/false` keyword clash that would defeat `ProB`.

$$ZBOOL ::= ztrue \mid zfalse$$

A metric's ordering direction. `dGeq` means larger is better ($\geq$ threshold), `dLeq` smaller is better ($\leq$), and `dEq` means closer to a fixed target is better ($=$).

$$Direction ::= dGeq \mid dLeq \mid dEq$$

The four outcomes a ratchet operation can produce. `pass` and `bootstrapPass` exit zero; `fail` and `failClosedError` exit non-zero. `bootstrapPass` is reserved for a genuine first adoption (no baseline anywhere); `failClosedError` is a controlled non-zero raised when git or the baseline blob is broken.

$$OUTCOME ::= pass \mid bootstrapPass \mid fail \mid failClosedError$$
$$\frac{}{exitCode : OUTCOME \rightarrow \mathbb{N}} \quad exitCode = \{pass \mapsto 0, bootstrapPass \mapsto 0, fail \mapsto 1, failClosedError \mapsto 1\}$$

The fail-closed characterisation of the exit code is fixed by the definition above: `exitCode o = 0` exactly when $o = pass$ or $o = bootstrapPass$. The two non-zero outcomes are the two failure kinds.

4 Metric Ordering

Each metric carries a direction and an absolute target. These are the `Thresholds` tables in the code (`oo_ratchet/thresholds.py`, `coupling/thresholds.py`). The model leaves the tables abstract: only the ordering they induce matters to the ratchet.

$$\begin{array}{l} \text{metricDir} : \text{METRIC} \rightarrow \text{Direction} \\ \text{metricTarget} : \text{METRIC} \rightarrow \mathbb{Z} \end{array}$$

Metric values are integers here. The code uses floats; **fuzz** has no reals, and the ratchet depends only on the *ordering* of values, never on their exact magnitude, so $\mathbb{Z}$ carries the model faithfully.

The absolute value, defined without unary minus so **fuzz** and ProB stay happy.

$$\begin{array}{l} \text{zabs} : \mathbb{Z} \rightarrow \mathbb{N} \\ \hline \forall n : \mathbb{Z} \bullet (n \geq 0 \Rightarrow \text{zabs } n = n) \\ \quad \wedge (n < 0 \Rightarrow \text{zabs } n + n = 0) \end{array}$$

The distance of a value from a metric’s target (used only by the **dEq** direction).

$$\begin{array}{l} \text{dist} : \text{METRIC} \times \mathbb{Z} \rightarrow \mathbb{N} \\ \hline \forall m : \text{METRIC}; v : \mathbb{Z} \bullet \text{dist}(m, v) = \text{zabs}(v - \text{metricTarget } m) \end{array}$$

“At least as good as”: the ratchet’s non-regression test.

$$\begin{array}{l} \text{betterEq} : \text{METRIC} \times \mathbb{Z} \times \mathbb{Z} \rightarrow \text{ZBOOL} \\ \hline \forall m : \text{METRIC}; c, b : \mathbb{Z} \bullet \text{betterEq}(m, c, b) = \text{ztrue} \\ \quad \Leftrightarrow (\text{metricDir } m = \text{dGeq} \wedge c \geq b) \\ \quad \vee (\text{metricDir } m = \text{dLeq} \wedge c \leq b) \\ \quad \vee (\text{metricDir } m = \text{dEq} \wedge \text{dist}(m, c) \leq \text{dist}(m, b)) \end{array}$$

“Strictly better”: the must-improve test.

$$\begin{array}{l} \text{strictBetter} : \text{METRIC} \times \mathbb{Z} \times \mathbb{Z} \rightarrow \text{ZBOOL} \\ \hline \forall m : \text{METRIC}; c, b : \mathbb{Z} \bullet \text{strictBetter}(m, c, b) = \text{ztrue} \\ \quad \Leftrightarrow (\text{metricDir } m = \text{dGeq} \wedge c > b) \\ \quad \vee (\text{metricDir } m = \text{dLeq} \wedge c < b) \\ \quad \vee (\text{metricDir } m = \text{dEq} \wedge \text{dist}(m, c) < \text{dist}(m, b)) \end{array}$$

“Meets its absolute threshold”: used to grade a brand-new file that has no baseline to improve against.

$$\begin{array}{l} \text{meets} : \text{METRIC} \times \mathbb{Z} \rightarrow \text{ZBOOL} \\ \hline \forall m : \text{METRIC}; v : \mathbb{Z} \bullet \text{meets}(m, v) = \text{ztrue} \\ \quad \Leftrightarrow (\text{metricDir } m = \text{dGeq} \wedge v \geq \text{metricTarget } m) \\ \quad \vee (\text{metricDir } m = \text{dLeq} \wedge v \leq \text{metricTarget } m) \\ \quad \vee (\text{metricDir } m = \text{dEq} \wedge v = \text{metricTarget } m) \end{array}$$

5 Metric Maps

A file’s metrics are a partial map from metric name to value; a baseline is a partial map from file to metric map.

$$\text{MMAP} == \text{METRIC} \rightarrow \mathbb{Z}$$

$$\text{BLINE} == \text{FILE} \rightarrow \text{MMAP}$$

The metrics of **c** that are worse than **b** — the regressed set. A metric absent from **b** never regresses (a new metric is not a regression).

$$\begin{array}{l} \text{worse} : \text{MMAP} \times \text{MMAP} \rightarrow \text{F METRIC} \\ \hline \forall c, b : \text{MMAP} \bullet \text{worse}(c, b) = \\ \quad \{m : \text{dom } c \mid m \in \text{dom } b \wedge \text{betterEq}(m, c \ m, b \ m) = \text{zfalse}\} \end{array}$$

The metrics of **c** that strictly improve on **b**.

$better : MMAP \times MMAP \rightarrow \mathbb{F} \text{ METRIC}$
$\forall c, b : MMAP \bullet better(c, b) =$ $\{m : \text{dom } c \mid m \in \text{dom } b \wedge strictBetter(m, c\ m, b\ m) = ztrue\}$

Whether every metric of a map clears its absolute threshold.

$allMeet : MMAP \rightarrow ZBOOL$
$\forall c : MMAP \bullet allMeet\ c = ztrue \Leftrightarrow$ $(\forall m : \text{dom } c \bullet meets(m, c\ m) = ztrue)$

6 The Shared Fail-Closed Core

Every check resolves a comparison base, reads the baseline committed *at that base commit*, and only then compares. The resolution and the fail-closed decisions that precede the comparison are identical across all three tools (`ratchet.py` for check-oo, `ratchet.py` for check-coupling, `baseline.py` for check-suppressions).

6.1 The git environment

`GitEnv` records what the resolution phase can observe: whether the base resolves, whether a baseline blob exists at the base and whether it is corrupt, the same for the `origin/main` tip, whether an in-tree baseline file is present, and whether `--require-base` was passed. The invariants encode the obvious well-formedness: a blob can be corrupt only if it exists, and can exist only if its ref resolves.

<i>GitEnv</i>
$baseResolves : ZBOOL$
$baseHasBlob : ZBOOL$
$baseCorrupt : ZBOOL$
$tipResolves : ZBOOL$
$tipHasBlob : ZBOOL$
$tipCorrupt : ZBOOL$
$inTreeExists : ZBOOL$
$requireBase : ZBOOL$
$baseCorrupt = ztrue \Rightarrow baseHasBlob = ztrue$
$baseHasBlob = ztrue \Rightarrow baseResolves = ztrue$
$tipCorrupt = ztrue \Rightarrow tipHasBlob = ztrue$
$tipHasBlob = ztrue \Rightarrow tipResolves = ztrue$

6.2 Base unresolvable

When no base commit can be found (`_no_base`): under `--require-base` the gate fails; a genuine first adoption — no in-tree baseline to protect — bootstraps; otherwise an in-tree baseline with an unresolvable base means a stale or unfetched `origin/main`, and the gate fails loudly rather than pass on trust.

<i>NoBase</i>
<i>GitEnv</i>
$out! : OUTCOME$
$baseResolves = zfalse$
$requireBase = ztrue \Rightarrow out! = fail$
$requireBase = zfalse \wedge inTreeExists = zfalse \Rightarrow$ $out! = bootstrapPass$
$requireBase = zfalse \wedge inTreeExists = ztrue \Rightarrow out! = fail$

6.3 Base resolves but carries no baseline blob

When the base commit has no baseline (`_absent_base_baseline`): a genuine first adoption requires the `origin/main` tip to also lack a baseline. A corrupt tip blob is a controlled error. A tip that carries a baseline means the branch forked before adoption and would launder a regression past the empty base, so it fails. An unresolvable tip with an in-tree baseline present cannot confirm first adoption, so it fails closed.

<i>AbsentBase</i> <i>GitEnv</i> <i>out!</i> : <i>OUTCOME</i>
$baseResolves = ztrue \wedge baseHasBlob = zfalse$ $tipCorrupt = ztrue \Rightarrow out! = failClosedError$ $tipResolves = zfalse \wedge inTreeExists = ztrue \Rightarrow out! = fail$ $tipResolves = zfalse \wedge inTreeExists = zfalse \Rightarrow$ $out! = bootstrapPass$ $tipResolves = ztrue \wedge tipCorrupt = zfalse \wedge tipHasBlob = ztrue$ $\Rightarrow out! = fail$ $tipResolves = ztrue \wedge tipHasBlob = zfalse \Rightarrow$ $out! = bootstrapPass$

6.4 Base baseline blob corrupt

A base baseline that resolves and exists but fails to parse — or is non-dict, carries a non-dict entry, holds a non-numeric or boolean metric, or is non-UTF8 — raises `GitError`, which the CLI turns into a controlled non-zero. All three tools reject these uniformly: check-oo's `show_baseline` (`oo_ratchet/gitio.py`) and its in-tree `Baseline._load` now type-harden the blob exactly as coupling and suppression do, so a parseable-but-ill-typed baseline can never reach the comparison and raise an uncaught `TypeError`.

<i>BaseError</i> <i>GitEnv</i> <i>out!</i> : <i>OUTCOME</i>
$baseResolves = ztrue \wedge baseHasBlob = ztrue \wedge baseCorrupt = ztrue$ $out! = failClosedError$

6.5 Reaching the comparison

The comparison runs only when the base resolves and carries a readable baseline blob. `Proceed` names that precondition; each per-tool check schema includes it.

<i>Proceed</i> <i>GitEnv</i>
$baseResolves = ztrue \wedge baseHasBlob = ztrue \wedge baseCorrupt = zfalse$

6.6 The CI-write guard

Every mutation (`update`, `relax`, `reconcile`, `rebaseline`) refuses to write under `GITHUB_ACTIONS` unless `--allow-ci-write` is passed. The guard is the same in all three tools.

<i>CiGuard</i> $githubActions : ZBOOL$ $allowCiWrite : ZBOOL$ $blocked! : ZBOOL$
$blocked! = ztrue \Leftrightarrow$ $(githubActions = ztrue \wedge allowCiWrite = zfalse)$

6.7 Fail-closed properties

The fail-closed contract is that `bootstrapPass` arises *only* from a genuine absence of baseline. These property schemas restate the guarantee as type-checked predicates over the transition schemas; each is a redundant strengthening of the schema it includes, so it has exactly the same models — it documents the property without weakening it.

<i>NoBaseFailClosed</i>
<i>NoBase</i>
$out! = bootstrapPass \Rightarrow inTreeExists = zfalse$

<i>AbsentBaseFailClosed</i>
<i>AbsentBase</i>
$out! = bootstrapPass \Rightarrow$ $(inTreeExists = zfalse$ $\vee (tipResolves = ztrue \wedge tipHasBlob = zfalse))$

7 check-oo: The Must-Improve Ratchet

The OO ratchet is the strict one. On the comparison path it requires, of a change that touches any file already tracked in the base baseline, that at least one metric *strictly* improve on such a file, and that no metric regress — unless a regression is waived by an audited relaxation.

7.1 The empty-baseline intercept

The OO ratchet shares the coupling guard against an empty `{}` base baseline (`ratchet.py`'s `_empty_baseline`). Under `--require-base` an empty base baseline — a truncated write or a bad merge — fails closed before the comparison, exactly like `CplEmptyRequire`; otherwise every touched file would look new and no regression could be seen. Without `--require-base` the empty baseline flows through the comparison, where the touched-file parse check and the in-tree lock and completeness checks still run.

<i>OoEmptyRequire</i>
<i>Proceed</i>
$baseCommit : COMMIT$
$baselineAt : COMMIT \leftrightarrow BLINE$
$baseB : BLINE$
$out! : OUTCOME$
$baseCommit \in \text{dom } baselineAt$ $baseB = baselineAt \ baseCommit$ $baseB = \emptyset$ $requireBase = ztrue$ $out! = fail$

7.2 The audited waiver

check-oo and check-coupling both admit an audited waiver, and admit it on identical terms; only check-suppressions — a single scalar — does not. A touched file's metric is *waived* (**RELAXED**) exactly when two conditions hold together: the in-tree baseline has locked the current value, so the regression was recorded rather than smuggled; and the audit log carries a matching `(file, metric)` relaxation recorded *since* the base commit, so a historical relaxation cannot bless a fresh regression. The waiver is per-metric: a relaxation names only the metrics that actually worsened (`writer.py`'s `relax` records only the regressed metrics), so relaxing one metric never waives another.

Waiver names this computation once, shared by both tools. It splits the raw regressions — the touched `(file, metric)` pairs worse than the base-commit baseline — into those a waiver covers (**waivedP**) and those it does not (**regressedP**). Only **regressedP** fails a gate.

Waiver

$inTree : BLINE$
 $current : BLINE$
 $baseB : BLINE$
 $touched : \mathbb{F} \text{ FILE}$
 $waivable : \mathbb{F}(\text{FILE} \times \text{METRIC})$
 $wvd : \mathbb{F}(\text{FILE} \times \text{METRIC})$
 $rawRegressed : \mathbb{F}(\text{FILE} \times \text{METRIC})$
 $regressedP : \mathbb{F}(\text{FILE} \times \text{METRIC})$
 $waivedP : \mathbb{F}(\text{FILE} \times \text{METRIC})$

$wvd = \{f : touched; m : METRIC \mid f \in \text{dom } inTree \wedge m \in \text{dom}(inTree f) \wedge inTree f m = current f m \wedge (f, m) \in$
 $rawRegressed = \{f : touched; m : METRIC \mid f \in \text{dom } baseB \wedge m \in worse(current f, baseB f)\}$
 $regressedP = rawRegressed \setminus wvd$
 $waivedP = rawRegressed \cap wvd$

The waiver is scoped to the change: **waivable** is the set of (**file**, **metric**) pairs harvested from audit relaxations recorded *since* the base commit (**audit.py**'s **relaxations_since**), never the whole audit history. A relaxation blessed on an earlier change is absent from this set, so it cannot waive a regression introduced now.

7.3 Inputs

OoCheck carries the resolved base commit and the base-commit baseline (**baselineAt baseCommit**, the authoritative comparison source — *never* the in-tree file); the current scores; the touched set (touched Python files that scored clean); and the touched files that failed to parse. The in-tree baseline, the waivable (**file**, **metric**) pairs, and the derived waived/regressed partition all come from the included **Waiver** schema — the same waiver check-coupling uses.

OoCheck

Proceed

Waiver

baseCommit : *COMMIT*

baselineAt : *COMMIT* $\rightarrow$ *BLINE*

brokenTouched : $\mathbb{F}$ *FILE*

missingF : $\mathbb{F}$ *FILE*

lockFailF : $\mathbb{F}$ *FILE*

basePresent : $\mathbb{F}$ *FILE*

improvedExisting : *ZBOOL*

newPasses : *ZBOOL*

improvementOk : *ZBOOL*

out! : *OUTCOME*

$baseCommit \in \text{dom } baselineAt$

$baseB = baselineAt \ baseCommit$

$\neg (baseB = \emptyset \wedge requireBase = ztrue)$

$touched \subseteq \text{dom } current$

$touched \cap brokenTouched = \emptyset$

$missingF = \{f : touched \mid f \notin \text{dom } inTree\}$

$lockFailF = \{f : touched \mid f \in \text{dom } inTree \wedge (\exists m : \text{dom}(current \ f) \bullet m \notin \text{dom}(inTree \ f) \vee inTree \ f \ m \neq current \ f)\}$

$basePresent = touched \cap \text{dom } baseB$

$improvedExisting = ztrue \Leftrightarrow$

$(\exists f : basePresent \bullet better(current \ f, baseB \ f) \neq \emptyset)$

$newPasses = ztrue \Leftrightarrow$

$(\exists f : touched \setminus \text{dom } baseB \bullet allMeet(current \ f) = ztrue)$

$improvementOk = ztrue \Leftrightarrow$

$waivedP \neq \emptyset$

$\vee (basePresent \neq \emptyset \wedge improvedExisting = ztrue)$

$\vee (basePresent = \emptyset \wedge newPasses = ztrue)$

$brokenTouched \neq \emptyset \Rightarrow out! = fail$

$brokenTouched = \emptyset \wedge touched = \emptyset \Rightarrow out! = pass$

$brokenTouched = \emptyset \wedge touched \neq \emptyset \Rightarrow$

$(out! = pass \Leftrightarrow$

$(missingF = \emptyset \wedge lockFailF = \emptyset$

$\wedge regressedP = \emptyset \wedge improvementOk = ztrue))$

$out! \neq bootstrapPass \wedge out! \neq failClosedError$

The waiver itself is modelled by **Waiver** above; **OoCheck** adds only the must-improve gate on top of it. That gate follows the code's S5: if any touched file is tracked in the base, the required improvement must come from such a file; a pure-add change is satisfied by a new file clearing all absolute thresholds; a change carrying a waiver ($waivedP \neq \emptyset$) is exempt.

8 check-coupling: The Regression-Only Ratchet

The coupling ratchet drops the must-improve gate, the lock-in check, and the completeness check, but keeps check-oo's audited waiver (**Waiver**). A touched file fails only if a metric is strictly worse than its base-commit baseline *and* the regression is not waived; holding steady passes, and so does a recorded, relaxation-blessed increase.

8.1 The empty-baseline intercept

Coupling shares the empty-baseline intercept with check-oo (**OoEmptyRequire**). An empty $\{\}$ base baseline (a truncated write or a bad merge) under **--require-base** fails closed before the comparison: otherwise every touched file would look new and no regression could be seen.

CplEmptyRequire
Proceed
baseCommit : *COMMIT*
baselineAt : *COMMIT* $\leftrightarrow$ *BLINE*
baseB : *BLINE*
out! : *OUTCOME*

baseCommit $\in$ dom *baselineAt*
baseB = *baselineAt* *baseCommit*
baseB = $\emptyset$
requireBase = *ztrue*
out! = *fail*

8.2 The comparison

The comparison runs when the base baseline is non-empty, or empty without `--require-base` (in which case every file is new and passes). A file absent from the base is new: it never regresses. The included **Waiver** supplies the same **regressedP** check-oo uses: a metric that is worse than the base but locked in the in-tree baseline and covered by a relaxation recorded since the base commit is waived, and so falls out of **regressedP**. Only the unwaived regressions fail the gate.

CplCheck
Proceed
Waiver
baseCommit : *COMMIT*
baselineAt : *COMMIT* $\leftrightarrow$ *BLINE*
brokenTouched : $\mathbb{F}$ *FILE*
out! : *OUTCOME*

baseCommit $\in$ dom *baselineAt*
baseB = *baselineAt* *baseCommit*
 $\neg$ (*baseB* = $\emptyset$ $\wedge$ *requireBase* = *ztrue*)
touched $\subseteq$ dom *current*
touched $\cap$ *brokenTouched* = $\emptyset$
brokenTouched $\neq \emptyset \Rightarrow$ *out!* = *fail*
brokenTouched = $\emptyset$ $\wedge$ *touched* = $\emptyset \Rightarrow$ *out!* = *pass*
brokenTouched = $\emptyset$ $\wedge$ *touched* $\neq \emptyset \Rightarrow$
 (*out!* = *pass* $\Leftrightarrow$ *regressedP* = $\emptyset$)
out! $\neq$ *bootstrapPass* $\wedge$ *out!* $\neq$ *failClosedError*

The waiver is why **CplCheck** still counts as *regression-only*: the verdict turns solely on **regressedP**, the raw regressions minus the waived ones. A waived metric is not a passing metric that was forced to improve — it is a recorded regression the audit log has blessed, so it never enters **regressedP** and never demands a compensating improvement.

9 check-suppressions: The Scalar-Count Ratchet

The suppression ratchet compares a single scalar: the total count of `noqa`, `type: ignore`, `pylint: disable`, `pyright: ignore` comments on code lines, plus the number of ruff `per-file-ignores` rule codes. The check fails only if the current total exceeds the base-commit total. The base total is coerced defensively (`_as_int`): a non-numeric, boolean, `NaN`, or infinite value becomes 0, which fails closed — a smaller base makes the current total register as an increase.

SupCheck

Proceed

baseCommit : *COMMIT*

supBaselineAt : *COMMIT* $\rightarrow \mathbb{N}$

baseTotal : $\mathbb{N}$

curTotal : $\mathbb{N}$

out! : *OUTCOME*

baseCommit $\in \text{dom } \text{supBaselineAt}$

baseTotal = *supBaselineAt* *baseCommit*

curTotal > *baseTotal* $\Rightarrow \text{out!} = \text{fail}$

curTotal $\leq$ *baseTotal* $\Rightarrow \text{out!} = \text{pass}$

out! $\neq \text{bootstrapPass} \wedge \text{out!} \neq \text{failClosedError}$

10 The Never-Loosen Update

--update tightens a baseline but never loosens it. The base ref scopes the touched set only; the regression test is against the *in-tree* baseline. check-oo and check-coupling share the per-metric algorithm (`apply.py`, `writer.py`); check-suppressions enforces the same guarantee on its single scalar total (`SupUpdate`).

The rename-aware base lookup: a file inherits its predecessor's entry when it is a rename target absent from the baseline under its new name, so a regression cannot launder itself through a rename. A file with no entry and no rename predecessor is new; the empty metric map models "no entry" (its *worse* set is empty, so a new file never regresses).

baseFor : *BLINE* $\times$ (*FILE* $\rightarrow$ *FILE*) $\times$ *FILE* $\rightarrow$ *MMAP*

$\forall b : \text{BLINE}; r : \text{FILE} \rightarrow \text{FILE}; f : \text{FILE} \bullet$

$(f \in \text{dom } b \Rightarrow \text{baseFor}(b, r, f) = b f)$

$\wedge (f \notin \text{dom } b \wedge f \in \text{dom } r \wedge r f \in \text{dom } b \Rightarrow \text{baseFor}(b, r, f) = b (r f))$

$\wedge (f \notin \text{dom } b \wedge \neg (f \in \text{dom } r \wedge r f \in \text{dom } b) \Rightarrow \text{baseFor}(b, r, f) = \emptyset)$

10.1 The update operation

The in-tree baseline is the state. A touched file is *writable* when it scored clean and does not regress against its rename-aware in-tree base; *refused* when it scored clean but regresses (its old entry is kept, so the carried base survives to the next run); dropped when it is gone from disk. Writing a rename target drops the stale source key. The write refuses under the CI guard, and fails the operation if anything was refused or unparseable.

NeverLoosenUpdate

inTree : *BLINE*
inTree' : *BLINE*
current : *BLINE*
touched : $\mathbb{F}$ *FILE*
renames : *FILE* $\bowtie$ *FILE*
parseErrTouched : $\mathbb{F}$ *FILE*
githubActions : *ZBOOL*
allowCiWrite : *ZBOOL*
guarded : *ZBOOL*
writable : $\mathbb{F}$ *FILE*
refusedF : $\mathbb{F}$ *FILE*
droppedMiss : $\mathbb{F}$ *FILE*
droppedSet : $\mathbb{F}$ *FILE*
updates : *BLINE*
out! : *OUTCOME*

guarded = *ztrue* $\Leftrightarrow$ (*githubActions* = *ztrue* $\wedge$ *allowCiWrite* = *zfalse*)
writable = $\{f : \textit{touched} \mid f \notin \textit{parseErrTouched} \wedge f \in \text{dom } \textit{current} \wedge \textit{worse}(\textit{current } f, \textit{baseFor}(\textit{inTree}, \textit{renames}, f))\}$
refusedF = $\{f : \textit{touched} \mid f \notin \textit{parseErrTouched} \wedge f \in \text{dom } \textit{current} \wedge \textit{worse}(\textit{current } f, \textit{baseFor}(\textit{inTree}, \textit{renames}, f))\}$
droppedMiss = $\{f : \textit{touched} \mid f \notin \textit{parseErrTouched} \wedge f \notin \text{dom } \textit{current}\}$
droppedSet = *droppedMiss* $\cup$ *ran*(*writable* $\triangleleft$ *renames*)
updates = *writable* $\triangleleft$ *current*
guarded = *ztrue* $\Rightarrow$ (*out!* = *fail* $\wedge$ *inTree'* = *inTree*)
guarded = *zfalse* $\Rightarrow$
 inTree' = (*droppedSet* $\triangleleft$ *inTree*) $\oplus$ *updates*
guarded = *zfalse* $\wedge$ *refusedF* = $\emptyset$ $\wedge$ *parseErrTouched* = $\emptyset$
 $\Rightarrow$ *out!* = *pass*
guarded = *zfalse* $\wedge$ (*refusedF* $\neq \emptyset \vee \textit{parseErrTouched} \neq \emptyset$)
 $\Rightarrow$ *out!* = *fail*
out! $\neq$ *bootstrapPass* $\wedge$ *out!* $\neq$ *failClosedError*

10.2 The never-loosen property

The guarantee: no metric of any file present both before and after the update is worse afterwards. This property schema restates it as a type-checked predicate over the operation.

NeverLoosens

NeverLoosenUpdate

$\forall f : \text{dom } \textit{inTree}' \cap \text{dom } \textit{inTree} \bullet$
 worse(*inTree'* *f*, *inTree* *f*) = $\emptyset$

10.3 The suppression update

check-suppressions' **update** enforces the same never-loosen guarantee on its single scalar total (**baseline.py**'s **_refuse_increase**). Past the CI guard, an update whose current total exceeds the in-tree baseline total is refused: nothing is written and the operation fails, exactly as the OO and coupling writers refuse a per-metric regression. A decrease or an unchanged total writes normally. The state is the in-tree total; the written total is never larger than the prior total.

SupUpdate

supTotal : $\mathbb{N}$
supTotal' : $\mathbb{N}$
curTotal : $\mathbb{N}$
githubActions : *ZBOOL*
allowCiWrite : *ZBOOL*
guarded : *ZBOOL*
out! : *OUTCOME*

guarded = *ztrue* $\Leftrightarrow$ (*githubActions* = *ztrue* $\wedge$ *allowCiWrite* = *zfalse*)
guarded = *ztrue* $\Rightarrow$ (*out!* = *fail* $\wedge$ *supTotal'* = *supTotal*)
guarded = *zfalse* $\wedge$ *curTotal* > *supTotal* $\Rightarrow$
 (*out!* = *fail* $\wedge$ *supTotal'* = *supTotal*)
guarded = *zfalse* $\wedge$ *curTotal* $\leq$ *supTotal* $\Rightarrow$
 (*out!* = *pass* $\wedge$ *supTotal'* = *curTotal*)

The never-loosen property for the scalar total: the written total is no larger than the prior total. This property schema restates it as a type-checked predicate over the operation.

SupNeverLoosens

SupUpdate

supTotal' $\leq$ *supTotal*

11 Findings

Modelling the code once surfaced three places where check-oo diverged from check-coupling and check-suppressions. All three were closed by the change that unified the base-baseline hardening, the empty-baseline intercept, and the suppression update, so the three tools now share a uniform contract:

- **Fail-closed base-baseline hardening.** All three blob readers reject a corrupt, non-dict, non-dict-entry, non-numeric, boolean, or non-UTF8 base baseline as a controlled non-zero (**BaseError**). check-oo's **show_baseline** and **Baseline._load** now type-harden the blob exactly as coupling and suppression do; no ill-typed-but-parseable baseline reaches the comparison.
- **Empty-baseline intercept.** All three fail closed on an empty `{}` base baseline under **--require-base** (**OoEmptyRequire**, **CplEmptyRequire**, and — for suppression — the coerced base total of 0 that any current count exceeds). check-oo no longer lets an empty base baseline flow into the comparison and pass a pure-add change as an ordinary **pass**.
- **Never-loosen update.** All three refuse to loosen on **--update**: check-oo and check-coupling refuse any per-metric regression; check-suppressions refuses any increase in the scalar total (**SupUpdate**, **SupNeverLoosens**).

12 System Invariants

The three properties the unified core guarantees, and where each lives in this model:

- **Fail-closed** — **bootstrapPass** arises only from a genuine absence of baseline: **NoBaseFailClosed**, **AbsentBaseFailClosed**, and the **exitCode** definition.
- **Never-loosen** — an update's written baseline is no worse on any metric: **NeverLoosens** (check-oo and check-coupling) and **SupNeverLoosens** (check-suppressions' scalar total).
- **Base-commit-authoritative** — the comparison reads *baselineAt baseCommit*, never the in-tree baseline: the equality *baseB* = *baselineAt baseCommit* in **OoCheck**, **CplCheck**, and (as a total) **SupCheck**. The in-tree baseline appears in **OoCheck** for the lock-in and waiver tests, in **CplCheck** (through **Waiver**) for the waiver test, and in **NeverLoosenUpdate** as the never-loosen target.